

P3 - Scalable Lakehouse Pipeline

PYSPARK + PARQUET + DELTA + FABRIC | PROJECT BOOK

Business story in beginner language

A logistics network emits many shipment events. You need a lakehouse that keeps raw history, cleans/validates events, derives trusted latest state/business outputs, handles incremental updates and avoids obvious Spark storage/performance mistakes.

What you are building

A Bronze/Silver/Gold lakehouse project with explicit schema, quality/quarantine, window/latest-state logic, Delta MERGE, partition/performance evidence and a Microsoft Fabric deployment plan.

What this project is really testing

- Distributed row/grain thinking.
- Explicit Spark schemas and quarantine.
- Bronze/Silver/Gold responsibilities.
- Delta MERGE/idempotency/schema evolution.
- Partition/shuffle/skew/small-file reasoning.
- Fabric mapping without fake cloud-runtime claims.

Important: teaching demo vs your protected project

The mini example below teaches the pattern using different toy data. It is NOT the final solution for your supplied project. After you understand the pattern, close the example and apply the reasoning to the project assets yourself.

Tiny teacher demonstration

Toy case: shipment events at event grain

A shipment may emit Created -> InTransit -> Delivered. Bronze keeps received events. Silver cleans/types and can derive the latest event per shipment. Gold may aggregate delivery performance. Those are three different grains.

Bronze grain: one received shipment event
Silver latest-state grain: one shipment
Gold daily metric grain: one hub per day
Do not sum or join across these layers until you can say which grain you are using.

Why partition by a unique ID is usually a trap

If every shipment_id is unique, partitioning by shipment_id can create thousands/millions of tiny folders/files. Choose a column with useful pruning and reasonable cardinality, such as event_date when workload commonly filters by date.

Candidate partition	Likely result
shipment_id	extremely high cardinality; many tiny partitions
event_date	often useful for date-bounded reads/backfills
hub_id	maybe useful only if workload and cardinality justify it

MERGE in plain English

An incremental event/update may match an existing business record. Delta MERGE lets you define matched update vs not-matched insert. Idempotency still depends on your key and ordering/update semantics.

STOP - explain before continuing

Why can an event_date partition help a historical backfill more than partitioning by unique shipment_id?

Say/write the answer in your own words. If you cannot, revisit the explanation before coding more.

Your project build order

Phase 1 - Profile event/ref data and define three grains

First understand

Bronze/Silver/Gold are useful only when you state what one row means at each layer.

Then do it in your project

1. Profile shipment_events + hub_reference + incremental/schema-evolution files.
2. Write event key/timestamp and malformed cases.
3. Write Bronze, Silver and Gold grains.

Evidence gate - prove before move-on

- source/event grain explicit
- latest-state key explicit
- known invalid rows identified

STOP - explain before continuing

Before Phase 2: can you explain what changed in the system and what evidence proves it?

Say/write the answer in your own words. If you cannot, revisit the explanation before coding more.

Phase 2 - Use explicit schema + quarantine

First understand

Spark inference can produce wrong types/null behavior; malformed rows need visible handling.

Then do it in your project

1. Define StructType/schema.
2. Parse dates/numbers explicitly.
3. Separate invalid/malformed rows with reasons.

Evidence gate - prove before move-on

- valid vs quarantined counts reconcile
- schema is explicit
- bad records are not silently dropped

STOP - explain before continuing

Before Phase 3: can you explain what changed in the system and what evidence proves it?

Say/write the answer in your own words. If you cannot, revisit the explanation before coding more.

Phase 3 - Build Bronze/Silver/Gold flow

First understand

Bronze preserves received data; Silver cleans/dedups/latest-state; Gold answers business query.

Then do it in your project

1. Write Bronze storage.
2. Create Silver cleaning/latest-state with window logic.
3. Join reference safely.
4. Build Gold output.

Evidence gate - prove before move-on

- each layer grain stated
- latest-state one row per shipment as designed
- Gold reconciles to Silver

STOP - explain before continuing

Before Phase 4: can you explain what changed in the system and what evidence proves it?

Say/write the answer in your own words. If you cannot, revisit the explanation before coding more.

Phase 4 - Choose Parquet/Delta and partition strategy

First understand

Storage format/partition are workload decisions, not labels to tick.

Then do it in your project

1. Write partition_plan.json with column + why.
2. Explain why shipment_id is rejected.
3. Document small-file mitigation.

Evidence gate - prove before move-on

- partition column not unique-id anti-pattern
- reason tied to filter/backfill workload
- small-file plan concrete

STOP - explain before continuing

Before Phase 5: can you explain what changed in the system and what evidence proves it?

Say/write the answer in your own words. If you cannot, revisit the explanation before coding more.

Phase 5 - Implement incremental Delta MERGE

First understand

MERGE key/order rule must handle redelivery/update safely.

Then do it in your project

1. Define matching key.
2. Implement matched update/not-matched insert.
3. Run same incremental data twice.

Evidence gate - prove before move-on

- logical latest-state unchanged on rerun
- updates applied once logically
- merge rule explained

STOP - explain before continuing

Before Phase 6: can you explain what changed in the system and what evidence proves it?

Say/write the answer in your own words. If you cannot, revisit the explanation before coding more.

Phase 6 - Inspect performance evidence

First understand

Spark performance comes from partitioning, shuffle, skew, file sizes and plan behavior - not random caching.

Then do it in your project

1. Inspect/explain plan/shuffle.
2. Use skew_events to reproduce skew concern.
3. Record one justified mitigation.
4. Explain when cache is unnecessary.

Evidence gate - prove before move-on

- performance.md names actual bottleneck
- skew plan tied to evidence
- no unsupported "faster" claim

STOP - explain before continuing

Before Phase 7: can you explain what changed in the system and what evidence proves it?

Say/write the answer in your own words. If you cannot, revisit the explanation before coding more.

Phase 7 - Design Fabric deployment

First understand

Map local design to OneLake/Lakehouse/notebook/pipeline/monitoring/identity without pretending local run is cloud proof.

Then do it in your project

1. Complete fabric_plan.md.
2. Map code/data layers to Fabric items.
3. Define identity/secret and monitoring approach.

Evidence gate - prove before move-on

- Fabric responsibilities clear
- cloud-only claims labeled plan/simulated unless executed
- identity/monitoring included

STOP - explain before continuing

Before Phase 8: can you explain what changed in the system and what evidence proves it?

Say/write the answer in your own words. If you cannot, revisit the explanation before coding more.

Phase 8 - Trigger schema/skew/small-file incidents

First understand

These failures test engineering judgment, not only syntax.

Then do it in your project

1. Save first failure evidence.
2. Repair schema handling.
3. Repair/mitigate skew.
4. Explain small-file compaction/optimization strategy.

Evidence gate - prove before move-on

- before/after evidence
- logical totals correct after repair
- performance trade-off documented

STOP - explain before continuing

Before Phase 9: can you explain what changed in the system and what evidence proves it?

Say/write the answer in your own words. If you cannot, revisit the explanation before coding more.

Phase 9 - Finish reproducibility + defense

First understand

A scalable design must still be understandable/replayable.

Then do it in your project

1. Complete README/runbook/performance/fabric plan.
2. Run checker/static validation possible.
3. Practice 10x defense.

Evidence gate - prove before move-on

- clean run instructions
- known limitations
- explain why Spark/Delta are justified here

STOP - explain before continuing

Before Phase complete: can you explain what changed in the system and what evidence proves it?

Say/write the answer in your own words. If you cannot, revisit the explanation before coding more.

Expected control evidence

Control	Expected from supplied starter/checker
source_rows	8
valid_rows	6
latest_shipments	5
quarantined_rows	2
partition	must not use shipment_id
Fabric plan	OneLake + Lakehouse + pipeline + monitoring
performance evidence	shuffle + skew + files discussed

Deliberate failures you must practise

Wrong/inferred schema

One or more events parse incorrectly.

Save first failing evidence before repair.

- explicit schema/change handling
- bad record visible
- trusted types preserved

Skewed join key

One key dominates and shuffle task becomes slow.

Save first failing evidence before repair.

- plan/task evidence
- justified mitigation such as broadcast/salting/repartition only when appropriate
- recheck correctness

Small files / bad partition

Too many tiny partitions/files hurt metadata/read efficiency.

Save first failing evidence before repair.

- reject unique-ID partition
- choose workload-aligned partition
- document compaction/OPTIMIZE strategy

Technical defense

What is the grain of Bronze vs Silver latest-state vs Gold?

Your answer: _____

Evidence you would point to: _____

Why did you choose your partition column and reject shipment_id?

Your answer: _____

Evidence you would point to: _____

What exactly makes the MERGE rerun-safe?

Your answer: _____

Evidence you would point to: _____

How did you diagnose skew instead of guessing?

Your answer: _____

Evidence you would point to: _____

Which parts did you truly run locally vs only map to Fabric?

Your answer: _____

Evidence you would point to: _____

What changes at 10x volume or stricter latency?

Your answer: _____

Evidence you would point to: _____

Honest limitations

A tiny local dataset cannot prove distributed performance at production volume. Use it to prove correctness and demonstrate the reasoning tools; state which scale/performance claims remain hypotheses until measured on representative data.

Evidence gate - prove before move-on

- README explains what is local/simulated.
- No claim of production scale without evidence.
- You can say what you would change at higher volume/availability/security requirements.